

High-Throughput Distributed Computing System

Enterprise-Scale Task Processing Architecture

Organization:	Fidelity Investments
Technology Stack:	FastAPI, Celery, Redis, Dask, AWS EC2
Throughput:	5TB/day data processing
Architecture Type:	Distributed Task Queue System

Executive Summary

Designed and implemented a production-grade distributed computing system capable of processing 5TB of data daily across multiple AWS EC2 instances. The architecture leverages FastAPI for high-performance API endpoints, Celery with Redis for distributed task orchestration, and Dask for parallel computation. This system serves enterprise-level workloads with fault tolerance, horizontal scalability, and optimized resource utilization.

System Architecture

The system implements a producer-consumer pattern with asynchronous task processing. Client requests are received through FastAPI endpoints, tasks are queued in Redis, distributed to worker nodes via Celery, and results are persisted in a backend storage system. This decoupled architecture ensures high availability and enables independent scaling of each component.

Core Components

Component	Technology	Purpose
API Layer	FastAPI	High-performance async HTTP endpoints for client requests
Message Broker	Redis	Task queue and inter-process communication
Task Orchestration	Celery	Distributed task queue and worker management
Computation Engine	Dask	Parallel and distributed computing framework

Infrastructure	AWS EC2	Scalable compute instances across availability zones
Result Storage	Redis/Database	Persistent storage for task results and metadata

Technical Implementation

Request Flow Architecture

The system processes requests through the following pipeline:

1	Client submits request to FastAPI endpoint
2	API validates request and enqueues task to Redis broker
3	Celery worker retrieves task from queue
4	Worker distributes computation across Dask cluster
5	Results stored in backend database
6	Client polls or receives webhook notification of completion

FastAPI Integration

FastAPI serves as the entry point for all client interactions, providing:

- Asynchronous request handling with Python `async/await` for non-blocking I/O
- Automatic request validation using Pydantic models
- OpenAPI documentation for API consumers
- Sub-50ms API response times for task submission
- Rate limiting and authentication middleware

Celery Task Distribution

Celery manages distributed task execution with the following configurations:

- Redis as both message broker and result backend for low-latency operations
- Multiple worker pools deployed across EC2 instances in different availability zones
- Task routing to specialized worker queues based on workload characteristics
- Automatic retry logic with exponential backoff for transient failures
- Worker autoscaling based on queue depth metrics

Dask Parallel Computing

Dask provides the computational backbone for CPU-intensive operations:

- Distributed `DataFrame` processing for large-scale data manipulation
- Task graph optimization to minimize data shuffling

- Integration with Celery workers for hybrid task execution
- Adaptive task scheduling based on worker resource availability
- Efficient memory management with lazy evaluation

Performance Characteristics

Metric	Value	Notes
Daily Throughput	5TB	Data processed per 24-hour period
Peak Request Rate	~1,200 req/sec	During market hours
Average Task Latency	2-15 seconds	Depends on computation complexity
API Response Time	<50ms	Task submission endpoint (p99)
Worker Instances	15-40 EC2 nodes	Auto-scaled based on load
Redis Operations	>10,000 ops/sec	Queue operations and result caching
System Uptime	99.7%	With graceful degradation

Scalability and Reliability

Horizontal Scaling Strategy

The system employs multiple scaling strategies to handle variable workloads:

- EC2 Auto Scaling Groups with custom CloudWatch metrics for worker fleet management
- Redis Cluster configuration for distributed caching and queue partitioning
- Load balancing across multiple FastAPI instances behind Application Load Balancer
- Stateless worker design enabling seamless instance additions and removals
- Resource-based task routing to optimize utilization across heterogeneous instances

Fault Tolerance Mechanisms

Built-in resilience features ensure system stability:

- Task acknowledgment and visibility timeout to prevent lost work
- Dead letter queues for failed tasks requiring manual intervention
- Circuit breakers for external service dependencies
- Health checks and automatic worker restart on failure
- Multi-AZ deployment for high availability
- Idempotent task design to safely support retries

Resource Optimization

Intelligent scheduling and resource management strategies maximize infrastructure efficiency:

- CPU and memory profiling to right-size EC2 instance types
- Task prioritization using Celery routing keys based on business criticality
- Spot Instance integration for non-critical background processing (40% cost reduction)
- Dask worker process pooling to minimize startup overhead
- Redis connection pooling and pipelining for reduced network round-trips
- Scheduled scaling policies aligned with business hour demand patterns

Monitoring and Observability

Comprehensive instrumentation provides real-time visibility into system behavior:

Component	Metrics Tracked
FastAPI	Request rate, latency percentiles, error rates, endpoint usage
Celery	Queue depth, task success/failure rates, worker utilization, retry counts
Redis	Memory usage, eviction rate, connection count, operation latency
Dask	Task execution time, memory per worker, scheduler queue size
Infrastructure	CPU/memory utilization, network throughput, disk I/O

Distributed tracing correlates requests across components, and structured logging enables rapid troubleshooting. Alerting thresholds trigger notifications for anomalies in queue depth, error rates, or latency degradation.

Technical Challenges and Solutions

Challenge	Solution Implemented
Task serialization overhead with large payloads	Implemented payload compression and reference-based task arguments stored in Redis
Memory leaks in long-running workers	Worker process lifecycle management with automatic recycling after N tasks
Uneven load distribution across workers	Custom task routing with consistent hashing and worker capability advertisement
Redis memory pressure during peak loads	Implemented TTL-based eviction, result streaming, and Redis Cluster sharding
Complex task dependencies and workflows	Celery Canvas primitives (chains, groups, chords) for workflow orchestration

Business Impact

The distributed computing system delivered measurable improvements to operational efficiency:

- Reduced batch processing time from 8 hours to 2 hours for critical daily workloads
- Enabled real-time analytics capabilities that were previously infeasible
- Improved infrastructure cost efficiency by 35% through optimized resource utilization
- Provided foundation for new product features requiring large-scale computation
- Achieved 99.7% system availability with graceful degradation during failures

Future Enhancements

Planned improvements to further scale and optimize the system:

- Kubernetes migration for improved orchestration and resource management
- GPU worker pools for machine learning inference workloads
- Enhanced observability with distributed tracing (OpenTelemetry)
- Serverless integration (AWS Lambda) for lightweight, event-driven tasks
- GraphQL API layer for flexible client data requirements
- Multi-region deployment for geographic load distribution

Conclusion

This distributed computing system demonstrates the effective application of modern Python frameworks and cloud infrastructure to solve enterprise-scale computational

challenges. By combining FastAPI's high-performance API layer, Celery's robust task distribution, Dask's parallel computing capabilities, and AWS's elastic infrastructure, the system achieves production-grade reliability while processing 5TB of data daily. The architecture's modular design ensures each component can be independently scaled and optimized, providing a solid foundation for future growth and evolving business requirements.